

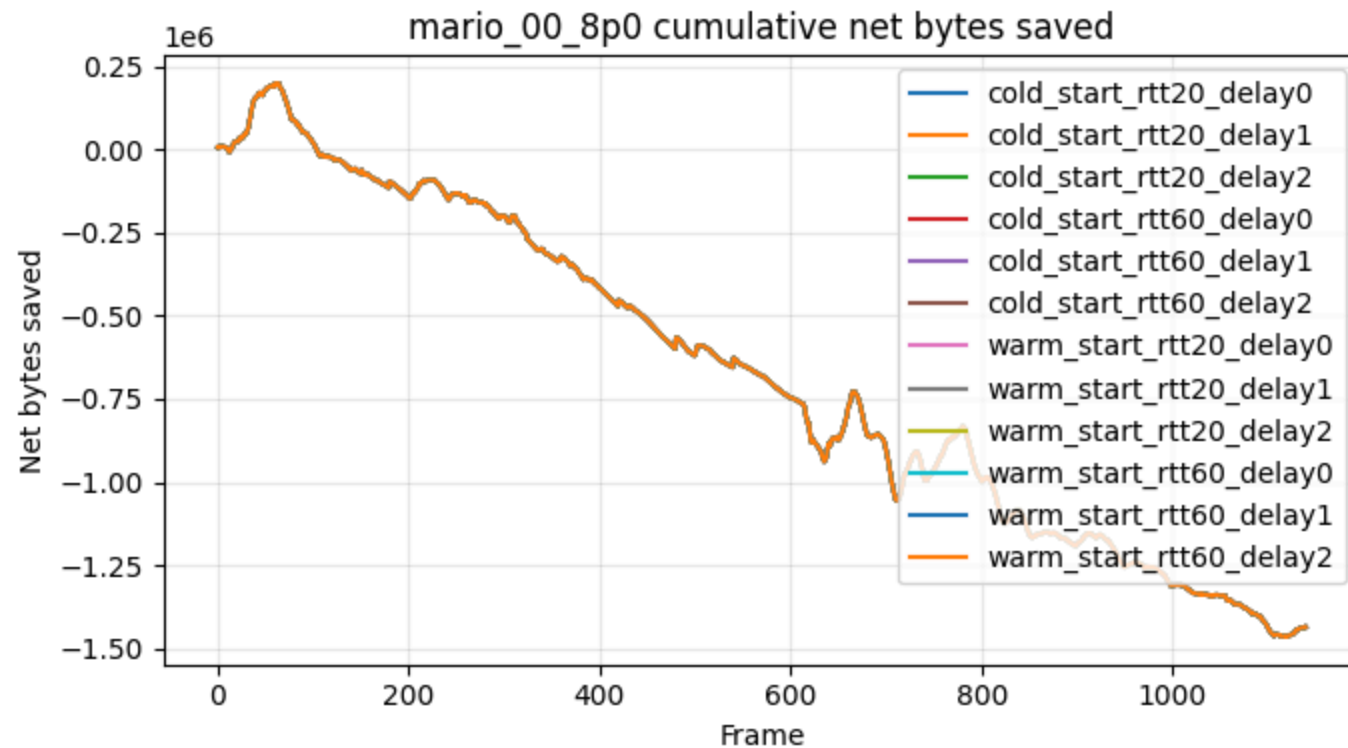
Exp3 + Exp5: Overhead and Generality

Conclusions

Question: when does RESPAWN save bandwidth after metadata overhead, and when does the method stop helping?

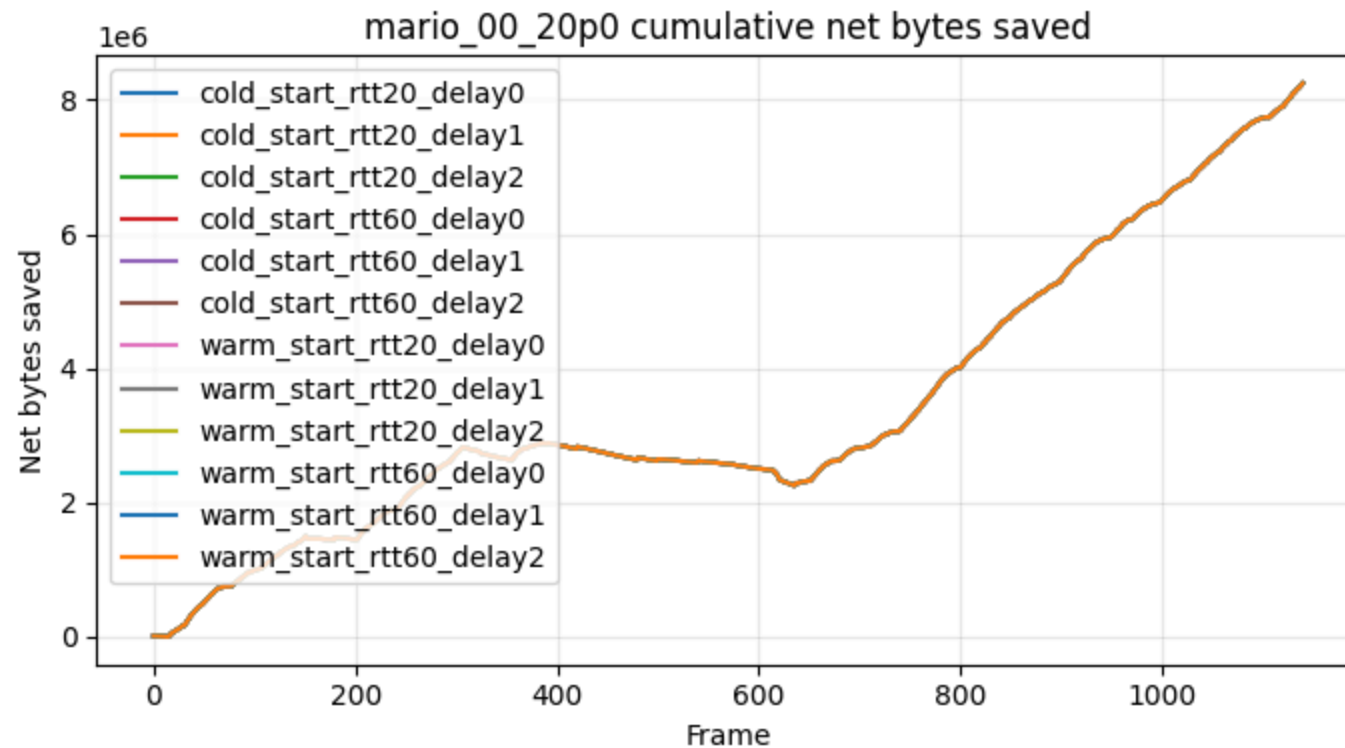
Main conclusion: RESPAWN helps most when recurring regions are large, persistent, and reusable. FM6 is the strongest case; Mario has high upside but a long negative tail; FC5 is mostly canceled by unstable content.

Exp3: Net Savings Depend on Bitrate Regime



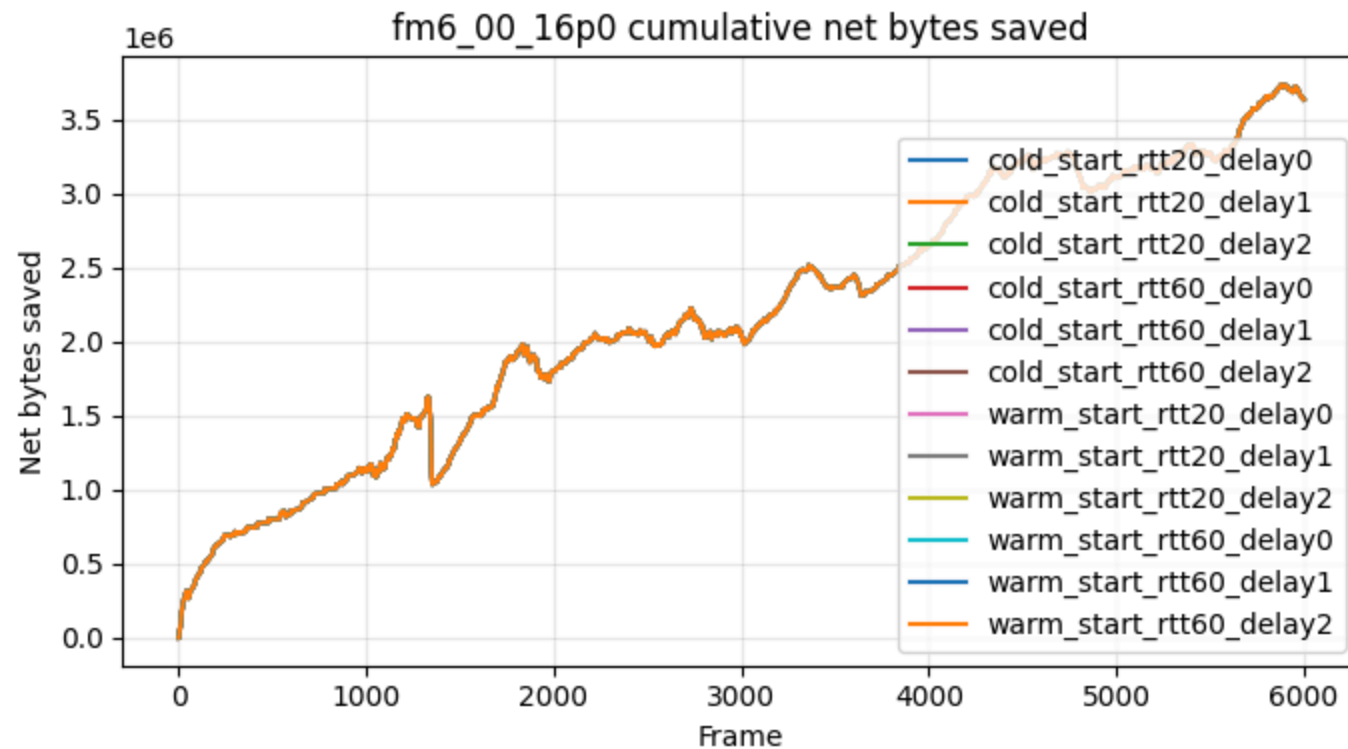
Caption: Mario at low bitrate. The pixel scene is already cheap for the baseline encoder, so the fixed MSK1 sidecar can outweigh video-byte savings.

Exp3: High Bitrate Can Amortize Sidecar Cost



Caption: Mario at higher bitrate. Baseline spends more bits preserving brick detail, while the masked stream stays cheaper; the fixed sidecar is amortized and net savings can turn positive.

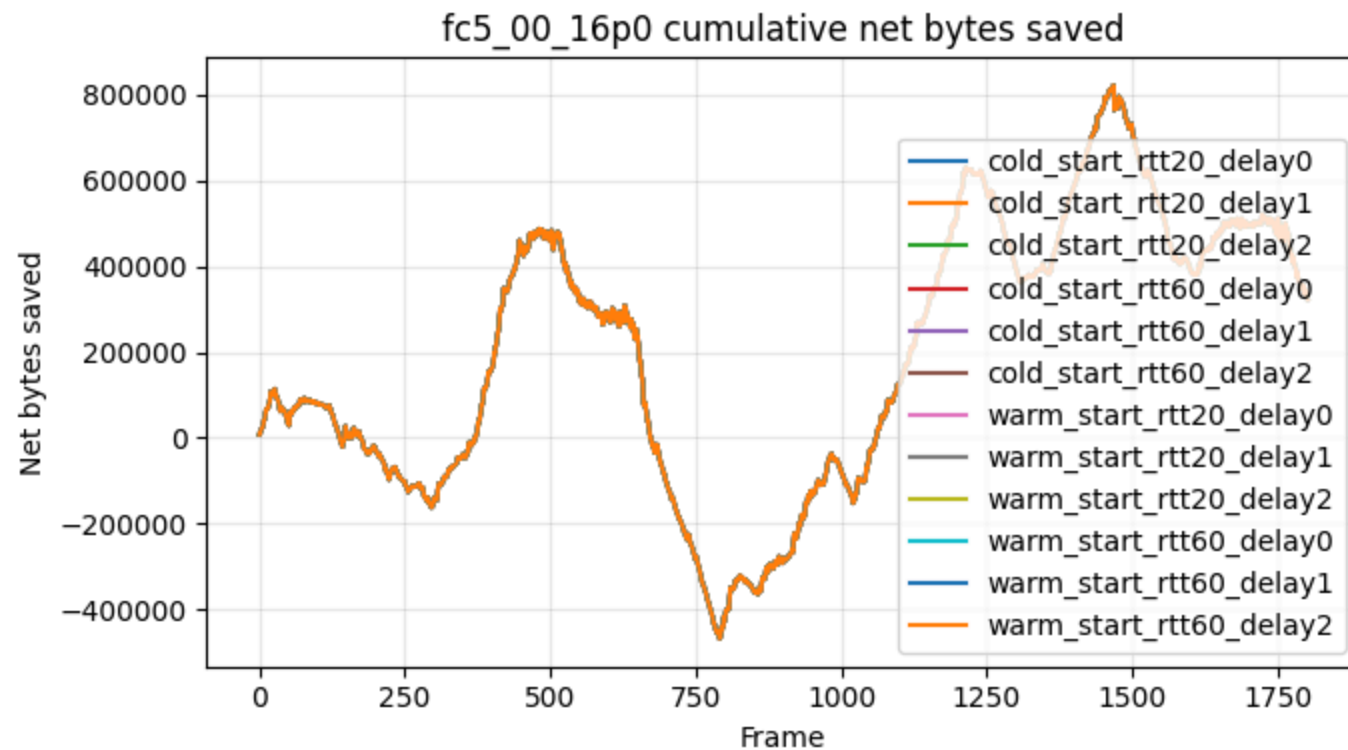
Exp3: Stable Photoreal Content Is Easier



Caption: FM6 has large, persistent, reusable regions and low ROI motion. This makes video-byte savings more consistent across GOPs and bitrates.

Takeaway: FM6 is the best current evidence that PESPAWN can produce stable net

Exp3: Unstable Content Cancels Gains



Caption: FC5 is clip-dependent and often near the break-even boundary. Even small changes in mask geometry, motion, and encoder decisions can flip savings negative.

Exp3 Caveat: Cache Curves Are an Upper-Bound Accounting View

Current Exp3 curves should be interpreted as:

```
baseline video bytes
- RESPAWN segmented video bytes
- MSK1 sidecar bytes
```

They are **not yet a full client template-delivery/cache-policy result**.

Reason: MSK1 payloads do not expose non-empty template IDs/paths for the simulator, so:

- `template_bytes_sent = 0`
- `forced_raw_frames = 0`
- cold/warm/partial-cache and RTT-delay variants collapse onto the same curve

Exp3 Proposal: Synthetic Template-Delay Sensitivity

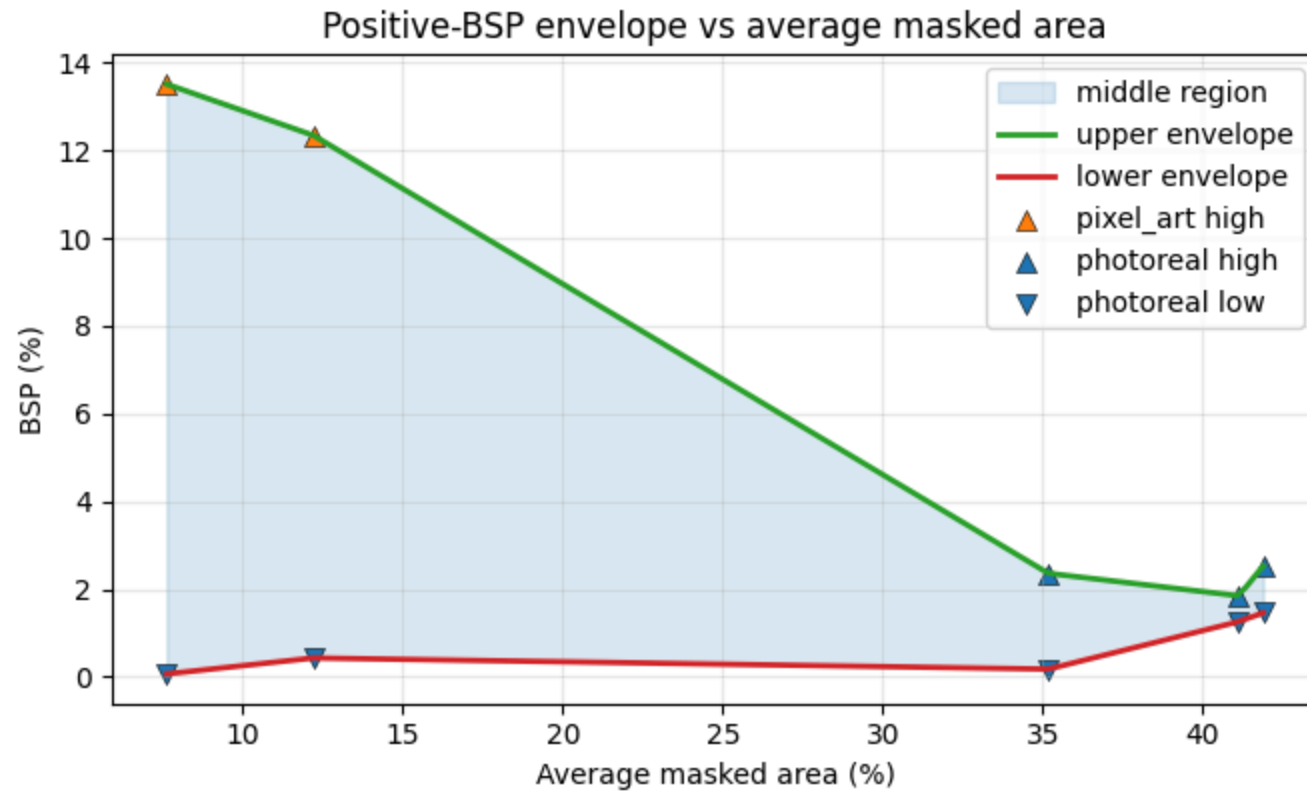
Until MSK1 carries real template IDs, we can still stress-test the system with a **simulation-only** model:

- assign synthetic template IDs from quantized region boxes
- use seeded probabilities for reuse, cache hit, and delivery loss
- charge synthetic template bytes when a template must be delivered
- delay template availability by `RTT x delay_rtts`
- force Raw while a needed template is unavailable

What this answers: how much cache miss, RTT delay, and template delivery overhead RESPAWN can tolerate before net savings disappear.

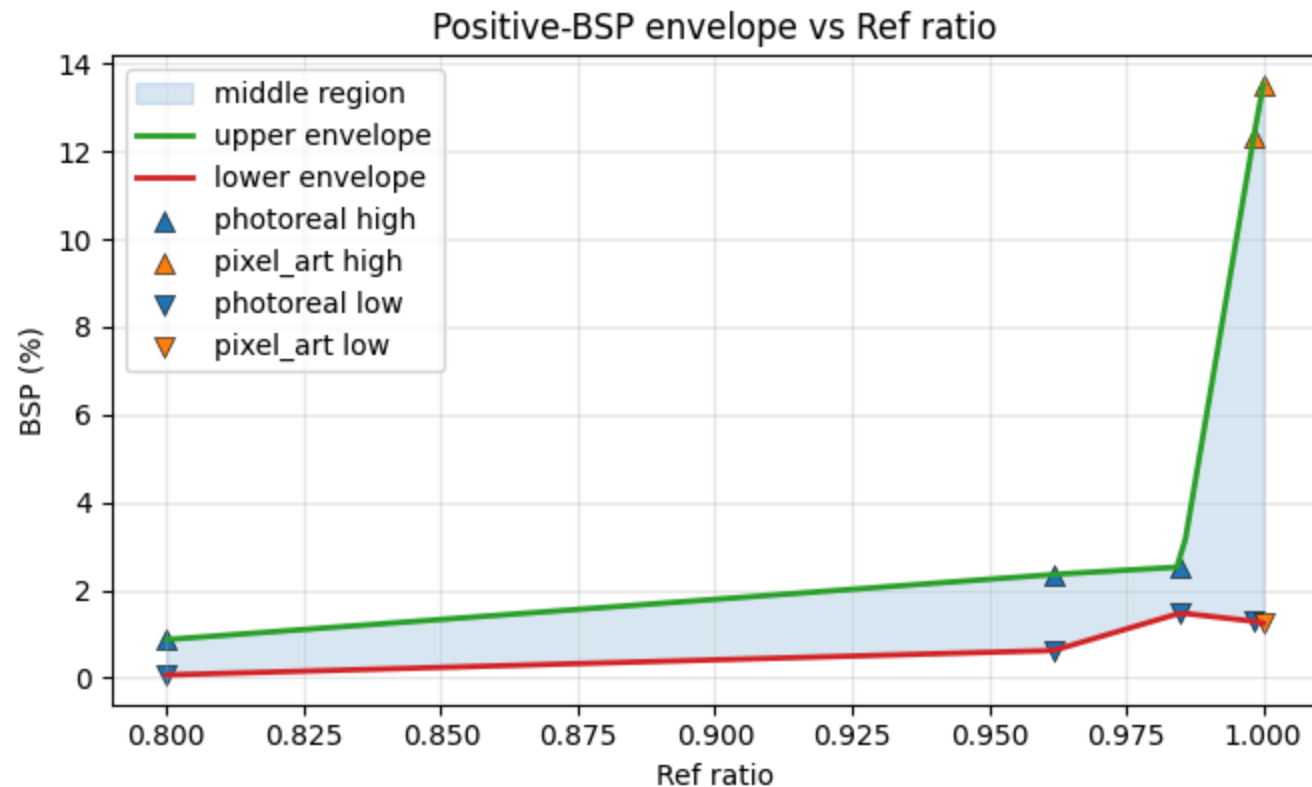
Labeling: these curves are sensitivity bounds, not measured client behavior

Exp5: Masked Area Explains Opportunity, Not Outcome



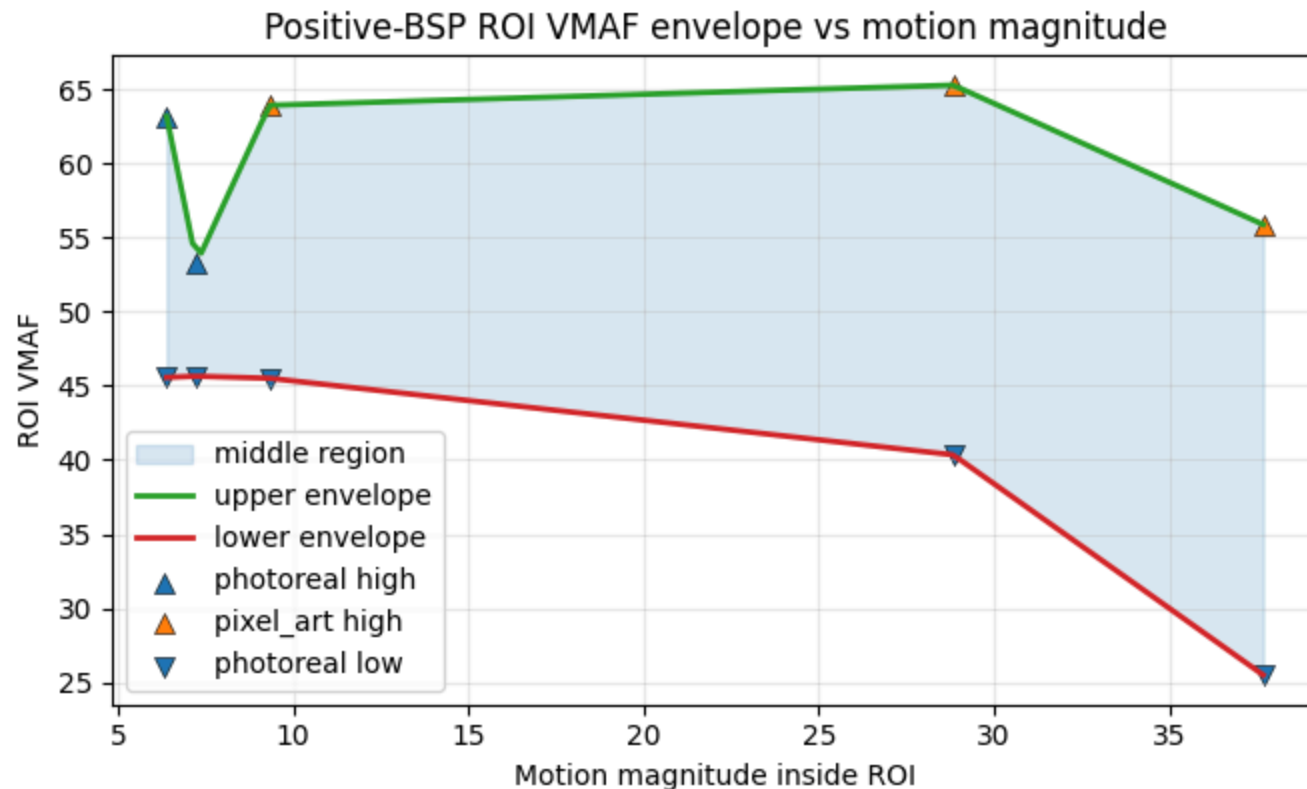
Caption: Positive-BSP envelope vs average masked area. Larger recurring regions

Exp5: Ref Ratio Is Necessary but Not Sufficient



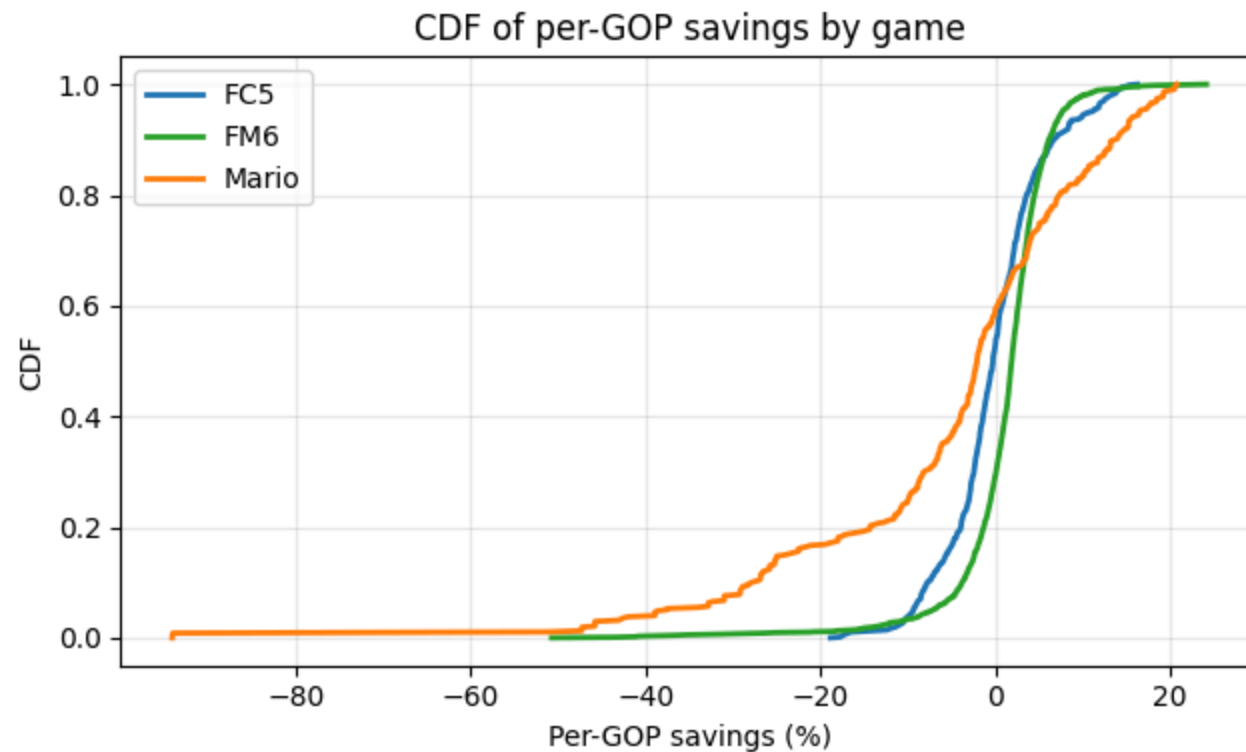
Caption: Positive-BSP envelope vs Ref ratio. High Ref ratio means the system often chooses template reuse, but it does not guarantee net savings.

Exp5: Motion and Appearance Drive Quality Risk



Caption: ROI VMAF envelope vs motion magnitude. Lower motion and stable appearance make reconstruction easier and less expensive.

Exp5: Per-GOP Savings Shows Workload Tails



Caption: CDF of per-GOP savings by game. This shows the distribution of savings rather than hiding failures behind an average.

Conclusion:

Combined Message for Paper Text

RESPAWN is strongest when a workload contains large, persistent, reusable regions. FM6 currently best matches this assumption and gives the cleanest net-savings story.

Mario shows that pixel-art content can produce strong savings, but only when bitrate is high enough to amortize sidecar overhead; otherwise the fixed metadata cost creates a long negative tail.

FC5 shows the boundary condition: if region appearance and motion are unpredictable, high Ref/Raw churn and unstable masks can cancel the compression benefit. This motivates future client-side template pool, healing, and stronger gating strategies.